

SmartAnalytics — Full Project Documentation

Project: `smartanalytics-app`

Version: 0.1.0

Last updated: July 2026

Stack: Next.js 15 · React 19 · Supabase (Postgres + Auth + Edge Functions) · GA4 Data API · Tailwind CSS

Table of Contents

1. [Executive Summary](#1-executive-summary)
 2. [Architecture Overview](#2-architecture-overview)
 3. [Repository Structure](#3-repository-structure)
 4. [Environment Variables](#4-environment-variables)
 5. [Authentication & Authorization](#5-authentication--authorization)
 6. [Database Tables](#6-database-tables)
 7. [Database Functions & RPCs](#7-database-functions--rpcs)
 8. [Edge Functions](#8-edge-functions)
 9. [Cron Jobs & Schedules](#9-cron-jobs--schedules)
 10. [Data Pipelines](#10-data-pipelines)
 11. [Inventory System](#11-inventory-system)
 12. [Frontend Application](#12-frontend-application)
 13. [API Routes](#13-api-routes)
 14. [Key Library Modules](#14-key-library-modules)
 15. [Deployment Checklist](#15-deployment-checklist)
 16. [Troubleshooting](#16-troubleshooting)
 17. [File Index](#17-file-index)
-

1. Executive Summary

SmartAnalytics is a dealer analytics platform for automotive, RV, powersports, and marine dealerships. It combines:

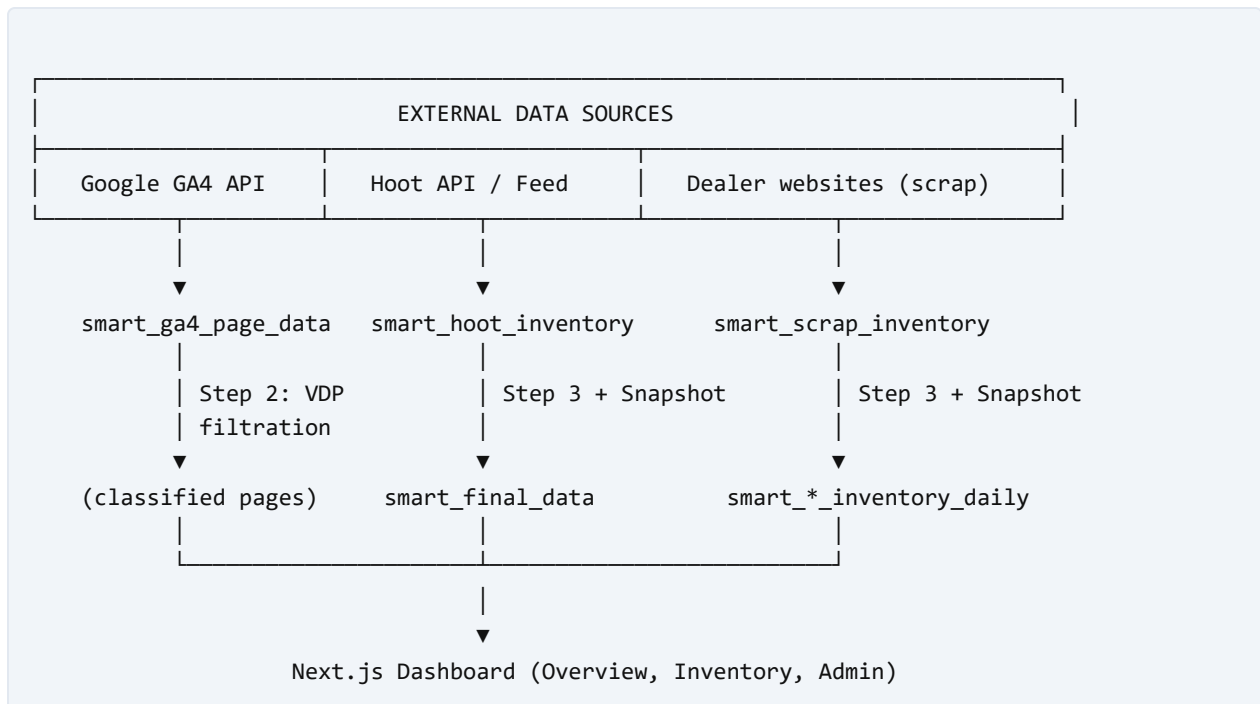
- **Google Analytics 4 (GA4)** page-view data
- **Hoot inventory** feeds (primary inventory source)
- **Web scraping** (fallback inventory for dealers without Hoot)
- **VDP classification** (regex rules per dealer/CMS)
- **Inventory-enriched reporting** (make, model, year, condition, location on VDP views)
- **Daily inventory snapshots** and a dedicated **Inventory Report** dashboard

Primary user flows:

User	What they do
Dealer	Sign in → pick dealer → view Overview, Inventory, Health
Superadmin	Admin login → manage dealers, VDP logics, run GA4/VDP pipeline, monitor daily sync

Identity key: `smart_hoot_config.ga4_customer_id = client_id` in all data tables = dashboard `ga4CustomerId`.

2. Architecture Overview



High-level data lifecycle

1. **Step 1 — GA4 sync:** GA4 API → `smart_ga4_page_data` (page-grain facts)

2. **Step 2 — VDP filtration:** Regex rules → classify pages (VDP/SRP/Home/Other)
3. **Step 3 — Final sync:** Join pages + inventory → `smart_final_data`
4. **Inventory snapshot:** Copy live inventory → daily tables (`pull_date`, `sk`)
5. **Dashboard:** RPCs read page/final/daily tables → charts, breakdowns, exports

3. Repository Structure

```
smartanalytics-app/
├─ public/                # Static assets
├─ src/
│  ├─ app/                # Next.js App Router (pages + API routes)
│  ├─ components/         # React UI components
│  ├─ hooks/              # Custom React hooks
│  ├─ lib/                # Business logic, API clients, pipeline
│  └─ middleware.js        # Route auth guards
├─ supabase/
│  ├─ migrations/         # Table DDL (run in SQL Editor)
│  ├─ rpc/                # Postgres functions (run in SQL Editor)
│  ├─ functions/          # Deno Edge Functions
│  ├─ cron/               # pg_cron schedule SQL
│  └─ scripts/             # One-off maintenance SQL
├─ workers/scrap-inventory/ # External scrap worker (Node.js)
├─ scripts/               # Dev utilities
├─ __tests__/             # Unit tests
├─ knowledge.md           # Technical KB (May 2026)
├─ Document.md            # This file
└─ package.json
```

4. Environment Variables

Next.js app (`.env.local`)

Variable	Scope	Required	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Browser + server	Yes	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Browser + server	Yes	Auth + client RPCs
`SUPABASE_SERVICE_ROLE_KEY`	Server only	Admin features	Pipeline, admin APIs
`GCP_SERVICE_ACCOUNT_JSON_PATH`	Server only	Step 1	Path to GA4 service account JSON
`GCP_SERVICE_ACCOUNT_JSON`	Server only	Alt.	Inline/base64 GA4 credentials

Scrap worker (`workers/scrap-inventory/.env`)

Variable	Purpose
`SUPABASE_URL`	Supabase URL
`SUPABASE_SERVICE_ROLE_KEY`	Call `upsert_scrap_inventory_batch`
`SCRAP_CLIENT_ID`	Optional single-dealer filter
`SCRAP_REPORT_DATE`	Optional `YYYY-MM-DD`
`SCRAP_CONCURRENCY`	Parallel dealers (default 2)

Edge Functions (auto-injected by Supabase)

- SUPABASE_URL
- SUPABASE_SERVICE_ROLE_KEY

Runtime modes

Condition	Behavior
Missing Supabase env	Demo mode (`demo@smartanalytics.dev`)
No service role	Dashboard works; admin pipeline returns 503
Full env	All features enabled

5. Authentication & Authorization

Dealer auth (Supabase Auth)

- Routes: `/login`, `/signup`
- Middleware: `src/middleware.js` protects `/dashboard/*`
- Session: Supabase cookies; inactivity timeout via `InactivityTimeout.jsx`

Superadmin auth

- Route: `/admin/login`
- Cookie-based superadmin session (`src/lib/auth/superadmin.js`)
- Required for `/dashboard/admin/*` and admin API routes

Route protection

Path	Who can access
`/dashboard`	Authenticated dealer

Path	Who can access
`/dashboard/admin/*`	Superadmin only
`/api/admin/*`	Superadmin + service role
`/api/dashboard/*`	Authenticated (server uses service role for RPCs)

6. Database Tables

6.1 Core tables (pre-existing in Supabase)

Table	Primary key / grain	Purpose
`smart_ga4_config`	Per dealer	GA4 property ID registry for Step 1
`smart_hoot_config`	Per dealer	Dealer master: name, `ga4_customer_id`, `hoot_url`, `is_active`
`smart_ga4_page_data`	Per page + date + client	GA4 page-grain facts after Step 1
`smart_ga4_data`	Per date + client	Session/user totals (optional KPIs)
`smart_vdp_logic`	Per dealer rule	VDP/SRP/home regex, `scrap_link`, `hoot_link`
`smart_hoot_inventory`	`sk`	**Live** Hoot inventory (external sync)
`smart_final_data`	Per VDP row	Inventory-enriched VDP facts (Step 3 output)

6.2 Tables from `supabase/migrations/`

Table	Migration	Purpose
`smart_scrap_inventory`	`smart_scrap_inventory.sql`	Live scrap inventory; PK `sk`
`smart_scrap_day_complete`	`smart_scrap_inventory.sql`	Scrape completion marker per dealer/day
`smart_scrap_run_log`	`smart_scrap_inventory_source.sql`	Scrape run audit
`smart_scrap_data`	`smart_scrap_data.sql`	Alternate scrap mirror
`smart_dealer_locations`	`smart_dealer_locations.sql`	Configured store locations
`smart_hoot_inventory_daily`	`smart_hoot_inventory_daily.sql`	Daily Hoot snapshot; PK `(pull_date, sk)`
`smart_hoot_inventory_daily_log`	`smart_hoot_inventory_daily.sql`	Hoot snapshot audit log
`smart_scrap_inventory_daily`	`smart_scrap_inventory_daily.sql`	Daily scrap snapshot; PK `(pull_date, sk)`
`smart_scrap_inventory_daily_log`	`smart_scrap_inventory_daily.sql`	Scrap snapshot audit log
`smart_user_login_sts`	`smart_user_login_sts.sql`	Login/session telemetry
`smart_ga4_page_data_backup`	`smart_ga4_page_data_backup.sql`	Backup table for migrations

6.3 Column migrations

Table	Migration	Columns added	
`smart_hoot_config`	`smart_hoot_config_pipeline_flags.sql`	`account_name`, `ga4_filter_enabled`, `final_vdp_enabled`	
`smart_hoot_config`	`smart_scrap_inventory_source.sql`	`inventory_source` (`hoot` \	`scrap`)

6.4 Key columns — `smart\_hoot\_inventory\_daily`

Column	Type	Description
`pull_date`	date	Business day captured (NOT `last_seen` date)
`sk`	text	Hoot stock key
`vin`, `make`, `model`, `year`	text	Vehicle attributes
`price`, `msrp`	numeric	Pricing
`condition`, `type_`, `location`	text	Classification
`first_seen`, `last_seen`	timestamptz	Hoot feed timestamps
`customer_name`, `advertiser`	text	Dealer linkage
`snapshotted_at`	timestamptz	When row was copied to daily table

6.5 Key columns — `smart\_scrap\_inventory\_daily`

Same as hoot daily plus:

Column	Description
`customer_id`	GA4 customer id (matches `ga4_customer_id`)

7. Database Functions & RPCs

Deploy all SQL files in `supabase/rpc/` via **Supabase SQL Editor** (run full file).

7.1 Admin VDP pipeline

Function	File	Purpose
`apply_vdp_filtration_range`	`apply_vdp_filtration_range.sql`	**Admin Step 2** — classify pages in exact date range
`apply_vdp_filtration`	`apply_vdp_filtration.sql`	**Cron Step 2** — lookback from today
`build_smart_final_data`	`build_smart_final_data.sql`	**Step 3 (Hoot)** — rebuild `smart_final_data`

Function	File	Purpose
`build_smart_final_data_scrap`	`build_smart_final_data_scrap.sql`	**Step 3 (Scrap)** — scrap inventory path
`build_date_wise_ga4_data`	`build_date_wise_ga4_data.sql`	Daily GA4 view totals
`build_date_wise_final_data`	`build_date_wise_final_data.sql`	Daily VDP views with filters
`build_date_wise_hoot_match`	`build_date_wise_hoot_match.sql`	Matched vs non-matched Hoot URL views
`build_pipeline_range_views`	`build_pipeline_range_views.sql`	Pipeline per-day coverage stats
`build_vdp_logics`	`build_vdp_logics.sql`	List/filter VDP logic rules
`get_scrap_dealers_for_sync`	`get_scrap_dealers_for_sync.sql`	Dealers with scrap enabled
`upsert_scrap_inventory_batch`	`upsert_scrap_inventory_batch.sql`	Batch upsert from scrap worker

7.2 Dashboard breakdowns & exports

Function	File
`get_ga4_channel_breakdown`	`get_ga4_channel_breakdown.sql`
`get_top_campaigns`	`get_top_campaigns.sql`
`get_location_breakdown`	`get_location_breakdown.sql`
`get_dealer_location_breakdown`	`get_dealer_location_breakdown.sql`
`get_make_breakdown`	`get_make_breakdown.sql`
`get_model_breakdown`	`get_model_breakdown.sql`
`get_year_breakdown`	`get_year_breakdown.sql`
`get_condition_breakdown`	`get_condition_breakdown.sql`
`get_type_breakdown`	`get_type_breakdown.sql`
`get_vdp_filter_options`	`get_vdp_filter_options.sql`
`get_vdp_views_total`	`get_vdp_views_total.sql`
`get_vdp_views_by_date`	`get_vdp_views_by_date.sql`
`get_all_dealers_channel_matrix`	`get_all_dealers_channel_matrix.sql`
`get_vdp_export_by_channel`	`get_vdp_export_by_channel.sql`
`get_vdp_export_by_location`	`get_vdp_export_by_location.sql`
`get_vdp_export_by_make`	`get_vdp_export_by_make.sql`
`get_vdp_export_by_model`	`get_vdp_export_by_model.sql`
`get_vdp_export_by_condition`	`get_vdp_export_by_condition.sql`
`get_all_tab_export`	`get_all_tab_export.sql`

7.3 Inventory snapshot & report

Function	File	Purpose
`snapshot_hoot_inventory_daily`	`snapshot_inventory_daily.sql`	Insert-only: live hoot → daily
`snapshot_scrap_inventory_daily`	`snapshot_inventory_daily.sql`	Insert-only: live scrap → daily
`snapshot_all_inventory_daily`	`snapshot_inventory_daily.sql`	Both sources
`run_daily_inventory_snapshot`	`snapshot_inventory_daily.sql`	Orchestrator (IST date, skip-if-complete)
`backfill_hoot_inventory_daily`	`smart_hoot_inventory_daily.sql`	Date-range backfill
`backfill_scrap_inventory_daily`	`smart_scrap_inventory_daily.sql`	Date-range backfill
`get_inventory_report`	`get_inventory_report.sql`	Full inventory report JSON
`con_inv_breakdown`	`con_inv_breakdown.sql`	Condition breakdown
`loc_inv_breakdown`	`loc_inv_breakdown.sql`	Location breakdown

7.4 Helpers & triggers

Function	File	Purpose
`extract_vin_from_text`	`extract_vin_from_text.sql`	VIN extraction
`page_path_matches_vdp_logic`	`page_path_matches_vdp_logic.sql`	Regex match helper
`inventory_matches_ga4_page_path`	`inventory_matches_ga4_page_path.sql`	URL/path inventory match
`smart_scrap_inventory_touch_last_seen`	`smart_scrap_inventory.sql`	Trigger: update `last_seen` on scrap update

7.5 RPCs used but not in repo (deploy separately)

Function	Used by
`get_ga4_overview`	Overview dashboard
`get_ga4_user_totals`	Overview KPIs (optional)
`next_dealer_to_sync`	Legacy edge sync
`apply_top_pages`	Legacy session-grain sync

7.6 `get\_inventory\_report` — detailed behavior

Signature:

```
get_inventory_report(  
  p_client_id  text,      -- NULL / '' / '__all_dealer__' = all dealers  
  p_report_date date,     -- exact pull_date
```



```

p_types      text[],
p_makes      text[],
p_models     text[],
p_locations  text[],
p_years      integer[],
p_condition  text      -- BOTH | NEW | USED
)

```

Data sources:

- smart_hoot_inventory_daily — hoot dealers
- smart_scrap_inventory_daily — scrap dealers / fallback

Dealer routing (per dealer):

1. hoot_link in VDP logic → hoot
2. scrap_link = on or URL → scrap
3. hoot_url or hoot daily rows → hoot
4. scrap daily rows → scrap
5. default → scrap

Hoot freshness rule:

- last_seen required within **24 hours** of pull_date (IST)
- **Today:** last_seen >= now() - 24 hours
- **Past dates:** last_seen on that IST calendar day
- Stale rows stay in DB but are **hidden** from report

Scrap: all rows for pull_date + dealer match (no last_seen filter)

Unit grain: one row per (dealer, sk) — matches Hoot API grain

Value: COALESCE(NULLIF(price,0), NULLIF(msrp,0), 0)

8. Edge Functions

Function	Path	RPC called	Schedule
`smart-master-sync`	`supabase/functions/smart-master-sync/`	`build_smart_final_data`	Hoot Step 3 cron
`smart-master-sync-scrap`	`supabase/functions/smart-master-sync-scrap/`	`get_scrap_dealers_for_sync`, `build_smart_final_data_scrap`	9–10 AM IST
`inventory-daily-snapshot`	`supabase/functions/inventory-daily-snapshot/`	`run_daily_inventory_snapshot`	10:30–11 AM IST

Deploy:

```
supabase functions deploy smart-master-sync
```

```
supabase functions deploy smart-master-sync-scrap
supabase functions deploy inventory-daily-snapshot
```

Manual invoke (inventory snapshot):

```
POST /functions/v1/inventory-daily-snapshot
Body: { "pull_date": "2026-07-10", "skip_if_complete": false }
```

9. Cron Jobs & Schedules

Requires `pg_cron` + `pg_net` in Supabase. Replace `__SERVICE_ROLE_KEY__` before running schedule SQL.

File	Job	UTC schedule	IST window	Target
<code>`smart-master-sync-scrap.schedule.sql`</code>	<code>`smart-master-sync-scrap`</code>	<code>`30,45 3 * * *`</code>	9:00–9:15 AM	Scrap Step 3
<code>`smart-master-sync-scrap.schedule.sql`</code>	<code>`smart-master-sync-scrap-2`</code>	<code>`0,15,30 4 * * *`</code>	9:30–10:00 AM	Scrap Step 3 retry
<code>`inventory-daily-snapshot.schedule.sql`</code>	<code>`inventory-daily-snapshot`</code>	<code>`0,15 5 * * *`</code>	10:30–10:45 AM	Daily inventory snapshot
<code>`inventory-daily-snapshot.schedule.sql`</code>	<code>`inventory-daily-snapshot-2`</code>	<code>`30 5 * * *`</code>	11:00 AM	Snapshot retry

External cron (not in repo):

- Hoot inventory feed → `smart_hoot_inventory`
- `workers/scrap-inventory` — daily ~6 AM

10. Data Pipelines

10.1 GA4 → Page data (Step 1)

```
POST /api/admin/pipeline/sync-page
  → ga4PageSync.js (GCP credentials)
  → smart_ga4_page_data
```

Key fields: `page_path`, `page_location`, `views`, `channel`, `report_date`, `client_id`

10.2 VDP filtration (Step 2)

```
POST /api/admin/pipeline/filtration
  → apply_vdp_filtration_range(client_id, from, to)
  → Updates smart_ga4_page_data: vdp_conditions, ga4_page_type, year, cms
```

Uses `smart_vdp_logic` regex patterns per dealer.

10.3 Final VDP sync (Step 3)

```
POST /api/admin/pipeline/final-sync
  → inventoryResolve.js picks hoot vs scrap RPC
  → build_smart_final_data OR build_smart_final_data_scrap
  → smart_final_data
```

Inventory resolution priority (`inventoryResolve.js`):

1. `hoot_link` → hoot RPC
2. scrap link or scrap inventory rows → scrap RPC
3. `hoot_url` or hoot inventory → hoot RPC
4. default → scrap RPC

10.4 Scrap sync

```
workers/scrap-inventory/run.mjs (cron)
  → get_scrap_dealers_for_sync
  → scrape dealer list pages
  → upsert_scrap_inventory_batch
  → smart_scrap_inventory
```

10.5 Inventory daily snapshot

```
pg_cron → inventory-daily-snapshot edge
  → run_daily_inventory_snapshot(IST today, skip_if_complete)
  → snapshot_hoot_inventory_daily (INSERT only, ON CONFLICT DO NOTHING)
  → snapshot_scrap_inventory_daily
  → smart_hoot_inventory_daily + smart_scrap_inventory_daily
```

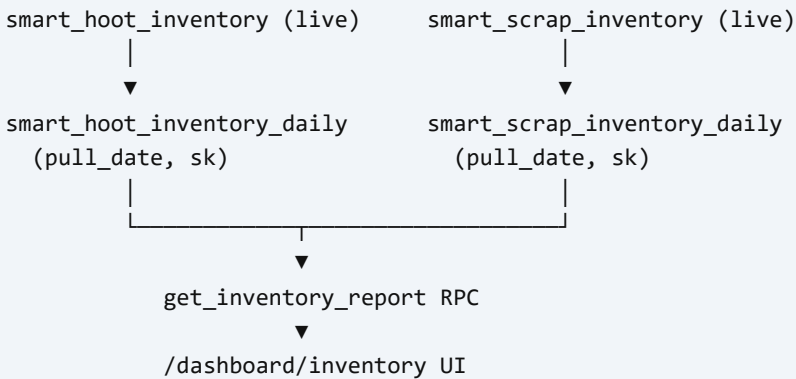
Re-run after deleting data:

```
DELETE FROM smart_hoot_inventory_daily_log WHERE pull_date = '2026-07-10';
```

```
DELETE FROM smart_scrap_inventory_daily_log WHERE pull_date = '2026-07-10';
SELECT public.run_daily_inventory_snapshot(DATE '2026-07-10', false);
```

11. Inventory System

11.1 Tables flow



11.2 Frontend inventory report

Module	Purpose
`InventoryReportContext.jsx`	State: date, filters, compare mode
`inventoryReport.js`	Fetch + exact date policy
`inventoryReportApi.js`	Calls `get_inventory_report` RPC
`inventoryReportNormalize.js`	Maps RPC JSON → UI shape
`InventoryReportFilters.jsx`	Filter bar (type, make, location, condition)
`InventoryBreakdownBlock.jsx`	Donut/bar breakdowns + compare
`InventoryList.jsx`	Unit list table

Exact date policy: If `meta.pullDate !== requestedDate`, UI shows zero (no fallback to older snapshot).

11.3 Useful SQL queries

Run snapshot (IST today):

```
SELECT * FROM public.run_daily_inventory_snapshot(
  (timezone('Asia/Kolkata', now()))::date,
  false
);
```

Count per pull_date:

```
SELECT pull_date, COUNT(*) FROM public.smart_hoot_inventory_daily
GROUP BY pull_date ORDER BY pull_date DESC LIMIT 14;
```

RPC unit count:

```
SELECT public.get_inventory_report(NULL, DATE '2026-07-10');
```

12. Frontend Application

12.1 Routes

Route	Page	Access
`/login`	Dealer sign-in	Public
`/signup`	Dealer sign-up	Public
`/admin/login`	Superadmin login	Public
`/dashboard`	Overview (GA4/VDP tabs)	Dealer
`/dashboard/inventory`	Inventory report	Dealer
`/dashboard/health`	Portfolio health	Dealer
`/dashboard/attribution`	Attribution (stub)	Dealer
`/dashboard/local`	Local intel	Dealer
`/dashboard/admin/pipeline`	3-step pipeline	Superadmin
`/dashboard/admin/dealers`	Dealer CRUD	Superadmin
`/dashboard/admin/vdp-logics`	VDP regex rules	Superadmin
`/dashboard/admin/daily-sync`	Daily sync matrix	Superadmin
`/dashboard/admin/date-wise-views`	GA4 date-wise report	Superadmin
`/dashboard/admin/ga4-daily`	GA4 daily panel	Superadmin

12.2 React contexts

Context	File	Role
`ClientContext`	`ClientContext.jsx`	Active dealer picker
`OverviewDataContext`	`OverviewDataContext.jsx`	Overview tabs, dates, filters

Context	File	Role
`AllDealerMatrixContext`	`AllDealerMatrixContext.jsx`	All-dealers channel matrix
`InventoryReportContext`	`InventoryReportContext.jsx`	Inventory report state
`ThemeProvider`	`ThemeProvider.jsx`	Light/dark theme

12.3 Overview dashboard tabs

Tab	Data source	Filters
All	`smart_ga4_page_data`	None
SRP	`smart_ga4_page_data`	`ga4_page_type = SRP`
Home	`smart_ga4_page_data`	`ga4_page_type = Home`
Other	`smart_ga4_page_data`	Remaining types
VDP	`smart_final_data`	Make, model, year, condition, location, type

13. API Routes

Dashboard (`/api/dashboard/\*`)

Method	Route	RPC / action
GET	`/overview`	`get_ga4_overview`, `get_ga4_user_totals`
GET	`/channel-breakdown`	`get_ga4_channel_breakdown`
GET	`/location-breakdown`	`get_location_breakdown`
GET	`/top-campaigns`	`get_top_campaigns`
GET	`/vdp-daily`	`build_date_wise_final_data`
GET	`/vdp-filter-options`	`get_vdp_filter_options`
GET	`/vdp-export`	5× export RPCs → XLSX
GET	`/all-export`	`get_all_tab_export`
GET	`/all-dealers-channel-matrix`	`get_all_dealers_channel_matrix`

Admin (`/api/admin/\*`)

Method	Route	Action
POST	`/login`	Superadmin session
GET/POST	`/dealers`	Dealer CRUD

Method	Route	Action
PATCH/DELETE	<code>`/dealers/[id]`</code>	Update/delete dealer
GET	<code>`/pipeline/dealers`</code>	List pipeline dealers
GET	<code>`/pipeline/stats`</code>	Pipeline coverage stats
POST	<code>`/pipeline/sync-page`</code>	Step 1 GA4 sync
POST	<code>`/pipeline/filtration`</code>	Step 2 VDP filtration
POST	<code>`/pipeline/final-sync`</code>	Step 3 final sync
GET/POST	<code>`/vdp-logics`</code>	VDP logic CRUD
POST	<code>`/vdp-logics/upload`</code>	CSV bulk upload
GET	<code>`/daily-sync`</code>	Daily sync status matrix
GET	<code>`/ga4-matrix`</code>	GA4 matrix data

Note: Inventory report calls `get_inventory_report` **directly from browser** via Supabase anon client (not a Next.js API route).

14. Key Library Modules

Supabase clients

- `src/lib/supabase/client.js` — browser anon
- `src/lib/supabase/server.js` — server component
- `src/lib/supabase/serviceRole.js` — admin/server RPCs

Pipeline

- `ga4PageSync.js` — Step 1 GA4 → Postgres
- `pipelineRpc.js` — Steps 2–3 wrappers
- `inventoryResolve.js` — hoot vs scrap Step 3
- `scrapSync.js` — in-app scrap helper
- `pipelineStats.js` — coverage stats

API / fetching

- `chunkedRpc.js` — date-chunked RPC (timeout safety)
- `overviewFetch.js` — overview bundle
- `dashboardApi.js` — dashboard RPC orchestration
- `inventoryReportApi.js` — inventory RPC client
- `vdpExport.js` / `allExport.js` — ExcelJS exports

Inventory

- `inventoryReport.js` — report orchestration
- `inventoryReportFilters.js` — filter → RPC params
- `inventoryReportNormalize.js` — RPC JSON → UI
- `inventoryReportPrefs.js` — localStorage dates

GA4

- `channelGroups.js` — channel rollup definitions
- `pageType.js` — page type → tab mapping
- `dateRange.js` — calendar date helpers (local TZ safe)

15. Deployment Checklist

Initial Supabase setup

1. Run all `supabase/migrations/*.sql` in SQL Editor (order: base tables first if missing)
2. Run all `supabase/rpc/*.sql` files
3. Deploy edge functions (`smart-master-sync`, `smart-master-sync-scrap`, `inventory-daily-snapshot`)
4. Run `supabase/cron/*.schedule.sql` (replace service role key)
5. Deploy RPCs not in repo: `get_ga4_overview`, `get_ga4_user_totals`

Daily operations (automatic)

Time (IST)	Job	Result
~6 AM	Scrap worker	<code>`smart_scrap_inventory`</code> updated
~9–10 AM	Scrap Step 3 cron	<code>`smart_final_data`</code> (scrap dealers)
~10:30–11 AM	Inventory snapshot cron	<code>`*_inventory_daily`</code> for today

Manual operations

```
-- Snapshot today (force re-run)
SELECT public.run_daily_inventory_snapshot(
  (timezone('Asia/Kolkata', now()))::date,
  false
);

-- Verify inventory report
SELECT public.get_inventory_report(NULL, CURRENT_DATE);
```


Next.js deploy

```
npm install
npm run build
npm start
```

Set all env vars on hosting platform (Vercel/etc.).

16. Troubleshooting

Symptom	Likely cause	Fix
Inventory shows 0 for today	No snapshot for IST today	Run <code>`run_daily_inventory_snapshot(..., false)`</code>
Snapshot skipped	Log says <code>`ok-insert-only`</code>	Delete log row or pass <code>`false`</code> for skip
22K units (stale data)	Freshness filter removed	Re-deploy <code>`get_inventory_report.sql`</code> with 24h rule
Hoot API $\neq$ report count	<code>`last_seen`</code> filter excludes stale	Expected; stale units hidden by design
Scrap zero, hoot has data	Dealer routed to hoot	Check <code>`smart_vdp_logic`</code> + <code>`hoot_url`</code>
Past date shows zero	<code>`last_seen`</code> not on that day	Normal for historical IST window
Admin pipeline 503	Missing <code>`SUPABASE_SERVICE_ROLE_KEY`</code>	Add server env var
GA4 sync fails	GCP credentials	Set <code>`GCP_SERVICE_ACCOUNT_JSON_PATH`</code>

17. File Index

Topic	Path
Technical KB	<code>`knowledge.md`</code>
Full documentation	<code>`Document.md`</code>
Inventory RPC	<code>`supabase/rpc/get_inventory_report.sql`</code>
Snapshot RPC	<code>`supabase/rpc/snapshot_inventory_daily.sql`</code>
Hoot daily migration	<code>`supabase/migrations/smart_hoot_inventory_daily.sql`</code>
Scrap daily migration	<code>`supabase/migrations/smart_scrap_inventory_daily.sql`</code>
Inventory UI	<code>`src/app/dashboard/inventory/page.jsx`</code>

Topic	Path
Pipeline admin	`src/components/dashboard/admin/DealerPipelineCard.jsx`
Scrap worker	`workers/scrap-inventory/run.mjs`
Auth middleware	`src/middleware.js`
Env template	`env.example`

End of SmartAnalytics Project Documentation